Diseño de interfaces web

UD 03. Modelo de cajas y posicionamiento CSS





IES Juan de Garay

Espai web de l'IES Juan de Garay.

Índice

1. Modelo de cajas ... 3
1.1. Contenido ... 4
1.2. Relleno ... 4
1.3. Borde ... 5
1.4. Margen ... 6
2. Selectores ... 6
2.1. Selector de etiqueta o elemento ... 7
2.2. Selector de clase ... 8
2.3. Selector de identificador ... 9
2.4. Combinators ... 10
2.5. Pseudoselectores ... 10
2.5.1. Pseudoclases ... 10
2.5.2. Pseudoelementos ... 11
2.6. La palabra clave !important ... 11
3. Unidades de medida en CSS ... 12
4. Posicionamiento ... 13
4.1. display ... 13
4.1.1. flex ... 14
4.1.2. grid ... 14
4.2. float ... 14
4.2.1. Primer ejemplo con float ... 14
4.2.2. Flotar contenedores ... 15
4.2.3. Flotar una caja a la izquierda ... 17
4.2.4. Flotar la Caja 1 y la Caja 2 a la izquierda ... 18
4.2.5. Flotar todas las cajas a la izquierda ... 19
4.2.6. Bloque contenedor demasiado estrecho ... 21
4.3. clear ... 22
4.4. position ... 24
4.4.1. static ... 25
4.4.2. relative ... 25
4.4.3. absolute ... 25
4.4.4. fixed ... 25
4.5. visibility ... 25
4.6. z-index ... 25

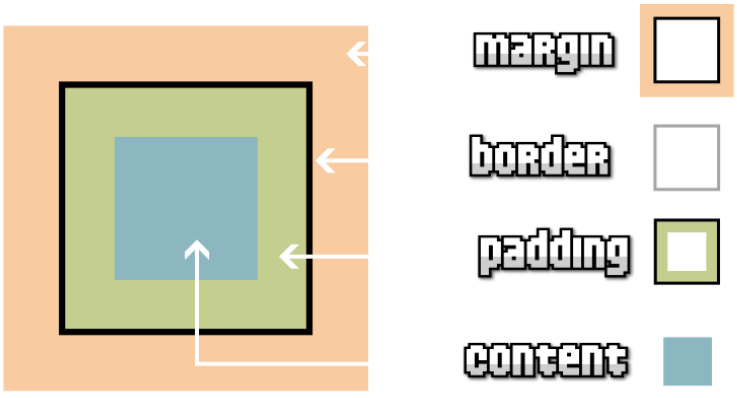
4.7. overflow 26
4.8. clip-path 27
5. CSS con tablas y listas 27
5.1. Cambiar apariencia de tablas 27
5.1.1. caption-side 27
5.1.2. table-layout 27
5.1.3. border-collapse 27
5.1.4. border-spacing 27
5.1.5. empty-cells 28
5.2. Cambiar apariencia de listas 28
5.2.1. list-style-type 28
5.2.2. list-style-image 29
5.2.3. list-style-position 29
5.2.4. list-style 29

1. Modelo de cajas

El modelo de cajas es un concepto fundamental para comprender el funcionamiento de las hojas de estilo. Aquí podrás ver una introducción básica a este modelo. De acuerdo con este modelo, todos los elementos de una página web generan una caja rectangular alrededor llamada “caja del elemento”.

En estas cajas se pueden configurar propiedades como bordes, márgenes y fondos (entre otras). Las cajas también se pueden emplear para posicionar los elementos y diseñar la página.

Las cajas de elementos, tal y como muestra la imagen, están hechas de cuatro componentes principales:



- **Contenido** (`content`), en azul, es la parte interior del elemento, excluyendo el relleno.
- **Relleno** (`padding`), en verde, es el espacio que rodea al contenido.
- **Margen** (`margin`), en naranja, es el espacio que rodea al borde, la parte más

externa del elemento.

- **Borde** (`border`), en negro, es el límite que separa el interior del exterior del elemento.

Hay algunas características fundamentales del modelo de cajas que vale la pena destacar:

- El relleno, los bordes y los márgenes son opcionales, por lo que, si ajustas a cero sus valores se eliminarán de la caja.
- Cualquier color o imagen que apliques de fondo al elemento se extenderá por el relleno.
- Los bordes se generan con propiedades de estilo que especifican su estilo (por ejemplo: sólido), grosor y color. Cuando el borde tiene huecos, el color o imagen de fondo aparecerá a través de esos huecos.
- Los márgenes siempre son transparentes (el color del elemento padre se verá a través de ellos).
- Cuando definas el largo de un elemento estás definiendo el largo del área de contenido (los largos de relleno, de borde y de márgenes se sumarían a esta cantidad).
- Puedes cambiar el estilo de los lados superior, derecho, inferior e izquierdo de una caja de un elemento por separado.

1.1. Contenido

Podemos definir el tamaño del contenido de la siguiente manera:

```
div {width:100px; height:200px; }
```

1.2. Relleno

Podemos definir el relleno. El orden de los valores son top (arriba), right (derecha), bottom (debajo) e left (izquierda), siempre en el sentido de las agujas del reloj y comenzando en las 12. Es fácil de recordar.

Establece un relleno para todos los elementos `div`:

```
div {padding: 3px 20px 3px 20px;}
```

Al omitir un valor, asume que el valor del relleno a la izquierda es el mismo que el de la derecha:

```
div {padding: 3px 20px 3px; }
```

Al omitir dos valores, asume que el primer valor corresponde al relleno por encima y por debajo del área de contenido y, el segundo valor corresponde al relleno a la derecha y a la izquierda:

```
div {padding: 3px 20px; }
```

Al omitir tres valores, asume que ese valor es el mismo para todos.

```
div {padding: 3px; }
```

Otras características interesantes del relleno son:

- El valor del relleno se sumará al de width ya definido en el elemento.
- Su color es el mismo al del área de contenido.
- El relleno nunca se "colapsa". Esto lo entenderás cuando veas los márgenes que sí se colapsan.

1.3. Borde

Los bordes funcionan, a la hora de establecer sus valores, de la misma manera que el relleno visto anteriormente, siguiendo un orden: superior, derecho, inferior, izquierdo.

Podemos configurar el estilo del borde, su grosor y su color.

Ejemplo de la propiedad `border-style` con la que configuramos el estilo del borde:

```
div {border-style: solid dashed dotted double; }
div {border-style: solid dashed dotted double; }
```

La propiedad `border-style` puede tener los valores: none, hidden, dotted, dashed, solid, double, groove, ridge, inset, outset e inherit. En el siguiente ejemplo configuramos cada uno de los lados de la caja con un borde distinto.

Ejemplo de la propiedad `border-width` con la que configuramos el grosor del borde:

```
div {
  border-style: solid;
  border-width: thin medium thick 12px;
}
```

Los valores que puede tomar esta propiedad son: `thin`, `medium`, `thick`, `inherit` o un valor concreto en píxeles. Si no se especifica esta propiedad tomará medium como valor por defecto. En el siguiente ejemplo configuramos un grosor distinto en cada uno de los lados del borde.

Ejemplo de la propiedad `border-color` con la que configuramos el color del borde:

```
div {
  border-style: solid;
  border-width: 4px;
  border-color: #333 #red rgb(0,0,255) #0044AC;
}
```

La propiedad `border` une todas las propiedades "border" vistas hasta ahora. En esta propiedad, a diferencia de las anteriores, no hay que colocar los valores en ningún orden concreto. La propiedad `border` se emplea cuando se quieren configurar los cuatro lados por igual. Aunque también tenemos las propiedades: border-top, border-right, border-bottom y border-left. Por ejemplo:

```
h1 {border: .5em solid blue; }
h2 {border-left: solid blue .5em; }
h3 {border-right: solid .5em; }
```

1.4. Margen

Los márgenes top y bottom de dos elementos que van seguidos se "colapsan". Es decir, se asume como margen entre ambos elementos el mayor de ellos. El siguiente ejemplo muestra lo que que ocurre cuando tenemos dos elementos un h1 y un h2 colocados uno a continuación del otro.

```
h1 {margin: 10px 20px 10px 20px; }
h2 {margin: 20px; }
```

En el primer caso el margen superior e inferior es de 10px. En el segundo caso es de 20px. El espacio resultante entre los dos elementos será de 20px.

Por el contrario, si fuesen dos elementos colocados "uno al lado del otro" (como dos elementos `span`), deberíamos tener en cuenta que los márgenes `right` y `left` no se colapsan, sino que se suman.

2. Selectores

Existen tres maneras básicas de referenciar elementos HTML desde CSS:

- Por etiqueta o elemento
- Por clase
- Por identificador

Aunque hay otras maneras más avanzadas de referenciar elementos HTML:

- Mediante combinators
- Por atributo
- Mediante pseudoselectores. Existen dos:
    - Pseudoclases
    - Pseudoelementos

2.1. Selector de etiqueta o elemento

Son aquellos que se aplican a un elemento (etiqueta) del lenguaje HTML. Por ejemplo:

CSS

```css
p {
  font-family: verdana;
  font-size: 20px;
}
```

HTML

```html
<p>Esto es un párrafo con estilo</p>
```

O:

CSS

```css
h1 {color: blue;}
h2 {color: blue;}
p {color: blue;}
```

Si te fijas en los ejemplos, verás que se está definiendo la misma propiedad (color) en todos los elementos e incluso se está asignando el mismo valor (blue). El ejemplo siguiente muestra como se puede escribir una única regla aplicada a varios selectores a la vez:

```css
h1, h2, p {color: blue;}
```

Existe un selector de elementos “universal” representado por el asterisco (*). El ejemplo siguiente muestra una regla que pondrá en gris todos los elementos del documento que no tengan especificado otro color.

```css
* {color: grey;}
```

2.2. Selector de clase

En HTML se emplea el atributo `class` para identificar distintos elementos como parte de un grupo conceptual. Así, los elementos de una clase pueden modificarse con una única regla de estilo.

En el siguiente ejemplo se muestra como dos elementos distintos se clasifican de la misma forma mediante la asignación del valor “especial” al atributo `class`.

CSS

```css
.bienvenida {
  color: white;
  text-align: center;
}
```

HTML

```html
<p class="bienvenida">Bienvenida con estilo</p>
```

También se puede hacer que un elemento pertenezca a más de una clase separando sus nombres de clase con espacios. En el siguiente ejemplo el párrafo pertenece a dos clases: “textocabecera” y “especial”.

CSS

```html
<p class="textocabecera especial">Hoy tenemos grandes rebajas.</p>
```

HTML

```html
<h1 class="especial">Atencion!</h1>
<p class="especial">Hoy tenemos grandes rebajas.</p>
```

Para escribir una regla de estilo que se aplique a todos los elementos de una determinada clase hay que escribir un punto seguido del nombre de la clase. Por ejemplo: .especial {color: green;}

El siguiente ejemplo muestra la forma de lograr que todos los elementos de la clase “especial” tengan un color verde a excepción de las cabeceras de primer nivel que tienen que ser rojas.

```css
.especial {color: green;}
h1.especial{color: red;}
```

2.3. Selector de identificador

Los elementos HTML disponen de un atributo llamado identificador (`id`), que tiene como finalidad identificar al de manera excluyente. De este modo, CSS u otro lenguaje podrá hacer referencia a él y distinguirlo del resto de los elementos del documento.

Se recomienda que el valor del `id` sea un nombre que caracterice o clarifique, de forma breve y esquemática al elemento y que, además, sea fácilmente reconocible por el programador. Se utilizan con frecuencia para identificar las secciones principales de las páginas: contenido, cabecera, pie, etcétera.

Para escribir una regla de estilo que se aplique a un determinado identificador hay que escribir el símbolo de la almohadilla (#) seguido del nombre del identificador. El ejemplo siguiente muestra algunas formas de establecer el tamaño de la fuente en 14 píxeles al elemento p identificado como “textocabecera” del ejemplo anterior:

CSS

```css
#bienvenida {
  color: black;
  text-align: center;
}
```

HTML

```html
<p id="bienvenida">Bienvenida con estilo</p>
```

Otro ejemplo:

CSS

```css
p#textocabecera {font-size: 14px;}
#textocabecera {font-size: 14px;}
```

HTML

```html
<p id="textocabecera">Texto de cabecera</p>
```

Con la primera regla indicamos que se aplique el estilo a un párrafo cuyo identificador sea “textocabacera” pero, como el id es único en cada documento, realmente basta con la segunda forma para decir lo mismo, porque no va a haber otro elemento `<p>` o diferente de `<p>` que tenga ese mismo identificador.

Los identificadores deben ser únicos para identificar de manera inequívoca el elemento. Si tenemos varios elementos que necesitan un tratamiento similar, emplearemos el atributo `class`.

2.4. Combinators

Al margen de la selección "básica" de elementos a través de CSS, que suele realizarse mediante clases e IDs, existe un amplio abanico de métodos para seleccionar elementos dependiendo de la estructura del documento HTML denominados combinadores CSS:

Nombre	Símbolo	Ejemplo	Significado
Agrupación de selectores	,	`p, a, div { }`	Se aplican estilos a varios elementos.
Selector descendiente	Espacio	`#page div { }`	Se aplican estilos a elementos dentro de otros.
Selector hijo	>	`#page > div { }`	Se aplican estilos a elementos hijos directos.
Selector hermano adyacente	+	`div + div { }`	Se aplican estilos a elementos que siguen a otros.
Selector hermano general	~	`div ~ div { }`	Se aplican estilos a elementos al mismo nivel.
Selector universal	*	`#page * { }`	Se aplican estilos a todos los elementos.

2.5. Pseudoselectores

Si queremos aplicar reglas de estilo a elementos especiales como: los vínculos visitados, la primera línea de un párrafo o su primera letra, emplearemos los pseudoselectores.

Hay dos tipos de pseudoelectores: pseudoclases y pseudoelementos.

2.5.1. Pseudoclases

Clasifican a los elementos basándose en características que van más allá de su nombre, atributos o contenido. La mayoría de las pseudoclases afectan a los elementos ancla (`a`), pudiendo definir un estilo diferente en función del comportamiento del enlace: si todavía no ha sido visitado (`:link`), si ya lo ha sido (`:visited`), mientras el ratón pasa por encima (`:hover`) o justo cuando se pulsa el ratón sobre él (`:active`).

Para emplear una pseudoclase se escribe la misma a continuación del selector separándola de éste por el símbolo de dos puntos (`:`). El siguiente ejemplo muestra cómo se distinguirían los cuatro estados posibles de un enlace mediante colores diferentes:

```
a:link {color: red;}
a:visited {color: blue;}
a:hover {color: fuchsia;}
a:active {color: maroon;}
```

Con el empleo de estas pseudoclases podemos quitar el subrayado de los hiperenlaces y hacer que aparezca sólo cuando pasamos el puntero por encima. Por ejemplo, siguiente muestra el empleo de la propiedad text-decoration para conseguir

dicho objetivo:

```
a:link {color: red; text-decoration: none;}
a:visited {color: blue; text-decoration: none;}
a:hover {color: red; text-decoration: underline;}
```

Debes tener en cuenta que las pseudoclases ancla deben aparecer siempre en un determinado orden. Este orden es: `:link, :visited, :hover` y `:active.`

Existen otras pseudoclases, que se emplean menos:

`:focus` hace referencia a los elementos que tienen el foco, como ocurre, por ejemplo, en los elementos de un formulario. Un ejemplo sería:

```
input:focus {background-color: yellow;}
```

`:first-child` hace referencia el primer hijo de un elemento padre. En el siguiente ejemplo se aplica el estilo al primer elemento de una lista desordenada:

```
ul li:first-child {font-weight: bold;}
```

`:lang` (idioma) hace referencia al idioma en el que está un determinado elemento. En el siguiente ejemplo se aplica el estilo a cualquier párrafo que esté escrito en inglés:

```
p:lang(en) {color:red}
```

2.5.2. Pseudoelementos

Estos pseudoelementos suelen ser partes de un elemento ya existente, como puede ser su primera línea `(:first-line)` o su primera letra `(:first-letter),` aunque también nos permite hacer referencia a elementos sin concretar en la estructura del documento porque dependen de la estructura del documento `(:before y :after).` Por ejemplo:

```
p:first-line {letter-spacing: 6pt;} /* añadimos espacio extra a la
primera línea del texto de cada párrafo */
p.definicion:first-letter {font-size: 300%; color: red;} /*
modificamos el estilo (tamaño y color) de la primera letra de los
párrafos pertenecientes a la clase "definicion" */
p.incompleto:after { content: " continuará ..."; } /* añadimos el
texto " continuará ..." al final de cada párrafo perteneciente a la
clase "incompleto" */
```

2.6. La palabra clave !important

`!important` se utiliza para dar prioridad a ciertas reglas. Cualquier definición de estilo que vaya acompañada de un !important tendrá prioridad sobre cualquier otra.

Como ya sabes, cuando tenemos una propiedad aplicada dos veces, el navegador hará caso a la última. En el ejemplo que se muestra a continuación está claro que se le asignarán 1200 píxeles al ancho del elemento identificado con el `id="principal".`

```
#principal {
  width: 800 px;
  width: 1200px;
}
```

Con esto de los selectores descendentes, hijos y adyacentes, se puede dar el caso de que, sin querer, estemos aplicando un estilo que no queremos a un elemento porque se ve indirectamente afectado por alguna de las reglas y no sabemos cuál. Por ejemplo: sale un párrafo en color rojo y no nos acordamos dónde lo configuramos porque realmente no tenemos ninguna regla que diga `p {color:red}`. En ese caso, y para no descerebrarnos buscando dónde está el fallo, si sabemos que los párrafos de texto emplean la letra de color negro, crearíamos la regla `p { color: green !important; }` ¡y solucionado!

> El uso de `!important` en CSS es algo que debe hacerse con cuidado, ya que aunque puede ser útil en ciertos casos, también puede generar problemas a largo plazo ya que rompe la cascada de CSS y hace más complicado el mantenimiento del código CSS.

3. Unidades de medida en CSS

Código	Unidad	Descripción	Uso en la Web
mm	Milímetros	Medidad métricas.	NO
cm	Centímetros		NO
in	Pulgadas		NO
pt	Punto	Unidad tradicional de medida equivalente a 1/72 pulgadas.	NO
pc	Pica	Unidad tradicional de medida equivalente a 12 puntos (o 1/6 pulgadas).	NO
px	Pixel	Las unidades de píxel son relativas a la resolución del monitor.	SI
em	Em	Medida relativa respecto al tamaño de la letra que por defecto mostraría el navegador.	SI
ex	Ex	Unidad de medida relativa que es la altura de la letra minúscula “x”.	SI
%	Porcentaje	El valor porcentual se refiere al tamaño por defecto aplicado en el navegador.	SI

La mejor solución de trabajo es configurar un tamaño por defecto en forma de porcentaje, y para cada elemento de la página configurar el tamaño con la unidad `em`.

Con:

```
body { font-size: 15px; }
```

El tamaño del texto variará según la resolución del monitor.

Con:

```
body { font-size: 200%; }
```

Se configura el tamaño del texto con respecto al tamaño configurado en el navegador. Si un navegador visualiza, por defecto, fuentes de 14 píxeles, un valor del 200% corresponde a 28 píxeles.

Con:

```
h1 { font-size: 2.5em; }
```

Establece el tamaño del texto de las cabeceras de primer nivel en un tamaño dos veces y medio mayor que el tamaño de la fuente mostrada por defecto en el navegador.

4. Posicionamiento

CSS utiliza el flotado y el posicionamiento para tener el máximo control sobre el lugar que ocupa cada elemento en una página web, sus condiciones de visibilidad y "flotabilidad", así como controlar el manejo de capas.

En los siguientes apartados veremos algunas propiedades de CSS que se utilizan para controlar el posicionamiento de los elementos. Estas son:

- `display`
- `float`
- `clear`
- `position`
- `visibility`
- `z-index`
- `overflow`
- `clip-path`

Un término que aparecerá a menudo es "flujo normal". Cuando hablamos de que los objetos de una página siguen el flujo normal del documento, queremos indicar que la forma en la que se disponen en la ventana del navegador coincide con el lugar que ocupan en el documento escrito (en el código HTML), donde el orden de lectura es de arriba a abajo y de izquierda a derecha.

Flotando y posicionando con CSS conseguimos que los elementos abandonen su flujo normal. De esta forma un elemento que este en el documento escrito más abajo que otro en el documento puede verse en el navegador por encima de él.

4.1. display

La propiedad `display` permite al documento interpretar de otra forma los elementos de tipo bloque y los elementos de tipo línea. Para ello basta con que asignes a esta propiedad el valor `block` si quieres que un elemento "en línea" se comporte como un elemento de tipo bloque y `none`, si quieres que un elemento de bloque no genere caja, no muestre su contenido y no ocupe espacio en la página.

4.1.1. flex

Leer → https://lenguajecss.com/css/flex/que-es-flex/

4.1.2. grid

Leer → https://lenguajecss.com/css/grid/que-es-grid/

4.2. float

La propiedad `float` sirve para mover una caja a la izquierda o a la derecha hasta que su borde exterior toque el borde de la caja que lo contiene o toque otra caja flotante.

Para que un elemento pueda flotar debe tener definido implícita o explícitamente su tamaño.

Las cajas flotantes no se encuentran en el "flujo normal" del documento por lo que las cajas que sí siguen el flujo normal se comportan como si las flotantes no estuviesen ahí. Es decir, no dejan el hueco de donde debería estar la caja flotante en un flujo normal, pero si se adaptan cuando se encuentran un elemento flotante.

La propiedad `float` puede tener los siguientes valores:

- `none` hará que el objeto no sea flotante.
- `left` hace que el elemento flote izquierda.
- `right` hace que el elemento flote a la derecha.
- `inherit` hará que el elemento tome el valor de esta propiedad de su elemento padre.

4.2.1. Primer ejemplo con float

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <ul>
    <li>Primer elemento</li>
    <li class="float">Segundo elemento</li>
    <li class="float">Tercer elemento</li>
    <li class="float">Cuarto elemento</li>
    <li>Quinto elemento</li>
    <li>Sexto elemento</li>
    <li>Séptimo elemento</li>
  </ul>
</body>
</html>
```

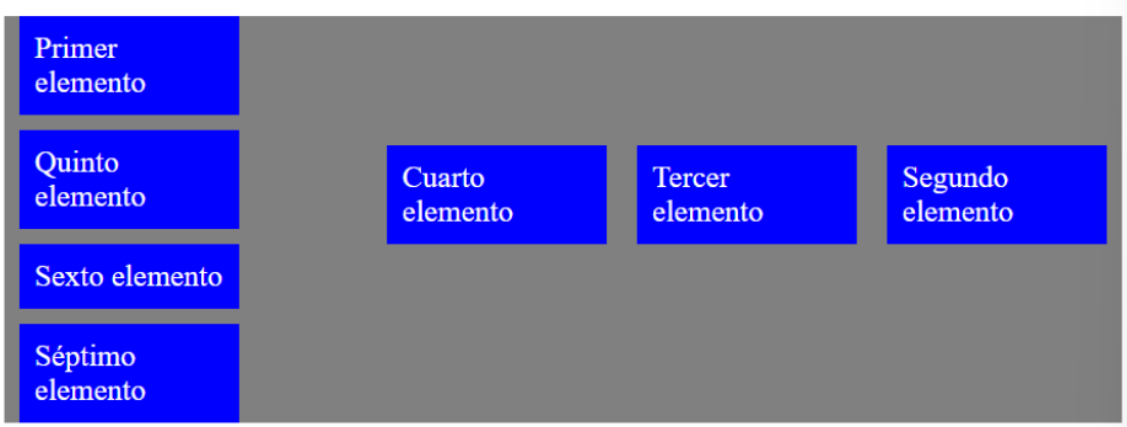
CSS:

```
ul {
  background-color: grey;
  padding: 0;
}

li{
  list-style-type: none;
    background: blue;
    width: 100px;
    padding: 8px;
    margin: 8px;
    color: white;
}

.float{
  float: right;
}
```

Resultado con `float:right`



Y si cambiamos `float:right` por `float:left`

4.2.2. Flotar contenedores

Para comprender la técnica de flotar cajas vamos a hacer este ejemplo. Se trata de tres contenedores (div) de distintos colores y tamaños, los tres están metidos en el interior de otro contenedor llamado “contenido” que lo que hace es conseguir que el conjunto se encuentre centrado en el navegador.

HTML:

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
    <head>
        <meta http-equiv="Content-Type" content="text/html; charset=iso-
8859-1" />
        <link rel="stylesheet" type="text/css" href="estilos.css" />
        <title>Flotar</title>
    </head>
```

```
    <body>
        <div id="contenido">
            <div id="caja1"> Caja 1 </div>
            <div id="caja2"> Caja 2 </div>
            <div id="caja3"> Caja 3 </div>
        </div>
    </body>
</html>
```

CSS:

```
body { font-weight: bold; color: white; }

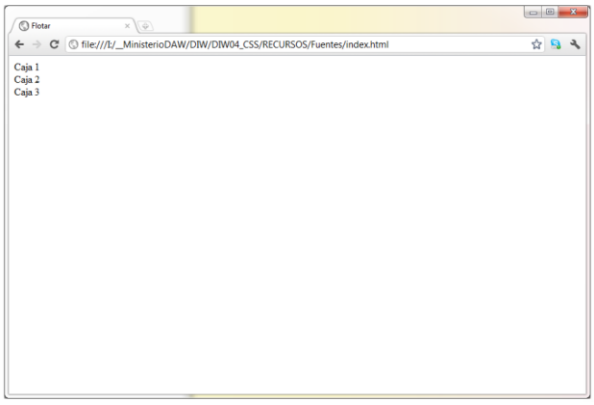
#contenido {
    /* Ancho de 700px y centrado en el navegador */
    width: 700px; margin: auto;
    /* Borde sólido y fondo gris claro */
    border: solid 1px black; background-color: #CCCCCC; }

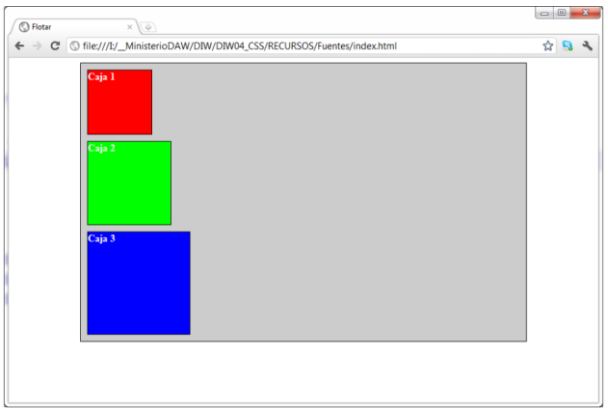
#caja1 {
    width: 100px; height: 100px; background-color:#FF0000;
    border: solid 1px black; margin: 10px; }

#caja2 {
    width: 130px; height: 130px; background-color:#00FF00;
    border: solid 1px black; margin: 10px; }

#caja3 {
    width: 160px; height: 160px; background-color:#0000FF;
    border: solid 1px black; margin: 10px; }
```

Página sin estilos y con estilos:





Como se trata de tres elementos de bloque cada uno de ellos comienza debajo del anterior.

Ahora flotamos la Caja1 a la derecha, así que tenemos que modificar el selector #caja1 del archivo estilos.css. El resto lo dejamos igual:

```
#caja1 {
    width: 100px; height: 100px;
    background-color:#FF0000;
    border: solid 1px black;
    margin: 10px;
    /*Flotar a la derecha */
```

```
    float: right;
}
```

Como se muestra en la siguiente imagen, cuando se flota la Caja1 a la derecha, se saca del flujo normal del documento y se mueve a la derecha hasta que su borde toca el borde derecho del bloque que lo contiene.



Recordar que la Caja 1 no llega a “pegar de todo” en el borde del contenedor pues en el estilo aplicado se configura, para las tres cajas, un margen de 10px en todos los lados.

4.2.3. Flotar una caja a la izquierda

Ahora vamos a flotar la Caja 1 a la izquierda, por lo que se saca otra vez del flujo del documento y se mueve a la izquierda hasta que el borde izquierdo del bloque toca el borde izquierdo del bloque que lo contiene.

Tenemos que modificar el selector #caja1 del archivo estilos.css. El resto lo dejamos igual:

```
#caja1 {
    width: 100px; height: 100px;
    background-color:#FF0000;
    border: solid 1px black;
    margin: 10px;
    /*Flotar a la izquierda */
    float: left;
}
```

La Caja 1 flotada a la izquierda quedaría así:



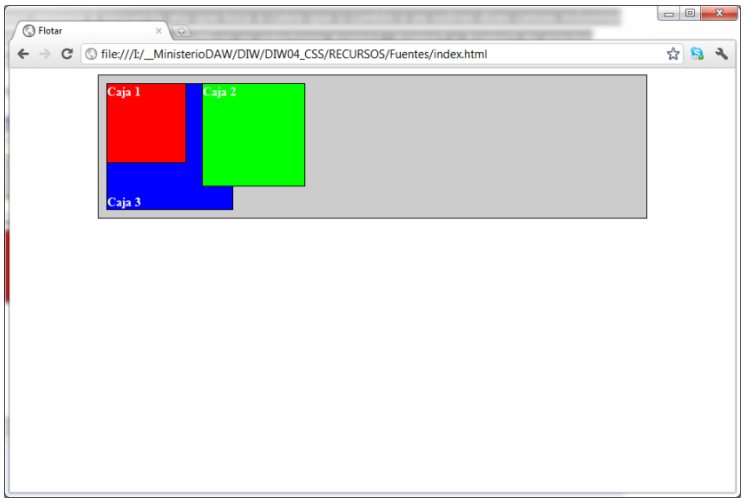
Puesto que la Caja ya no se encuentra en el flujo del documento, no ocupa espacio y se sitúa sobre la Caja 2, ocultando así parte de ella.

4.2.4. Flotar la Caja 1 y la Caja 2 a la izquierda

Ahora vamos a hacer que flote también la Caja 2. Para conseguir esto, tenemos que configurar el selector `#caja2` del archivo estilos.css tal y como se hizo en el apartado anterior para la Caja 1.

```
#caja2 {
    width: 130px; height: 130px;
    background-color: #00FF00;
    border: solid 1px black;
    margin: 10px;
    /*Flotar a la izquierda*/
    float: left;
}
```

Flotando las Cajas 1 y 2 a la izquierda la web quedaría así:



Al flotar ahora también la Caja 2, ésta se mueve hasta que toca su borde izquierdo con el borde derecho de la Caja 1. Esto es así porque la Caja 1 también está flotada y ambas fueron sacadas del flujo normal del documento.

La Caja 3, junto con el Contenedor, es la única que queda en el flujo normal del documento.

Debemos fijarnos que el tamaño del Contenedor se adapta al de los elementos que siguen en el flujo normal.

4.2.5. Flotar todas las cajas a la izquierda

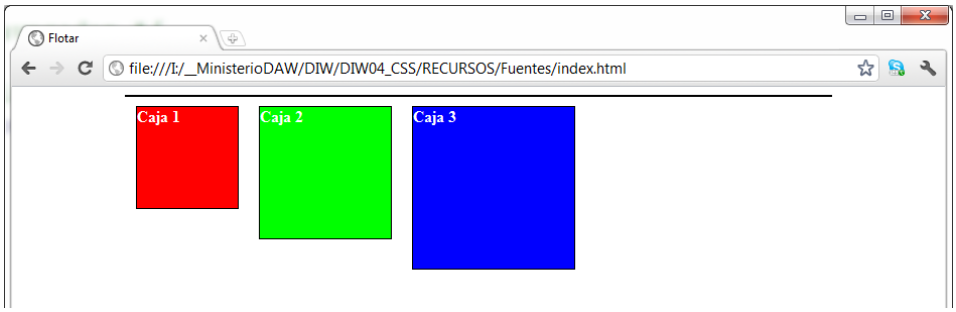
Al flotar las tres cajas a la izquierda:

- La Caja 1 se mueve a la izquierda hasta que toca el borde izquierdo del contenedor que lo contiene.
- La Caja 2 se mueve también a la izquierda hasta que toca la Caja 1.
- La Caja 3 también se mueve a la izquierda hasta que toca con la Caja 2.

Esto es así porque todas las Cajas salen del flujo normal del documento y se ven entre sí.

Para conseguir esto, tenemos que modificar los selectores #caja1, #caja2 y #caja3 del archivo estilos.css tal y como se hizo en los apartados anteriores.

```
#caja3 {
    width: 160px; height: 160px;
    background-color:#0000FF;
    border: solid 1px black;
    margin: 10px;
    /*Flotar a la izquierda*/
    float: left;
}
```



Tal y como se ve la imagen, el div “Contenido” tiene ahora una altura de 0px, ya que todo lo que tiene en su interior está fuera del flujo del documento. Es decir, como no tiene nada “dentro” no necesita tener una altura determinada.

Los divs de las cajas de colores siguen estando “en el interior” del div “contenido” y eso se nota en que no salen del ancho marcado por éste y que su alineamiento es con respecto a él.

Para que se vuelva a ver el “Contenido” tenemos varias opciones:

- Se le puede aplicar una altura configurando en el selector `#contenido` la propiedad `height` con una altura mayor que el alto de la caja más grande.

- Se puede flotar a la izquierda la caja con id `contenido`. De esta forma volvería a aparecer pero flotada a la izquierda de su contendedor (`body`).
- Añadir, dentro del div “contenido” y a continuación de las tres cajas flotadas, un `div` totalmente vacío. Este `div` lo configuramos estableciendo la propiedad `clear` con el valor `both` (aunque en este caso también serviría `left`, pues todas las cajas flotadas anteriormente lo están a la izquierda). Esta es la solución más utilizada y la que emplearemos.

Veamos cómo quedan los códigos de los archivos HTML y CSS después de realizar la última operación explicada en la diapositiva anterior de modo que todas las cajas flotan a la izquierda y se ve igualmente el “contenido”.

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">

<head>
    <meta http-equiv="Content-Type" content="text/html; charset=iso-8859-
1" />
    <link rel="stylesheet" type="text/css" href="estilos.css" />
    <title>Flotar</title>
</head>

<body>
    <div id="contenido">
        <div id="caja1"> Caja 1 </div>
        <div id="caja2"> Caja 2 </div>
        <div id="caja3"> Caja 3 </div>
        <!-- Caja vacía para limpiar flotados -->
        <div class="clearboth"></div>
    </div>
</body>

</html>
```

CSS:

```
body { font-weight: bold; color: white; }

#contenido {
    /* Ancho de 700px y centrado en el navegador */
    width: 700px; margin: auto;
    /* Borde sólido y fondo gris claro */
    border: solid 1px black; background-color: #CCCCCC;}

#caja1 {
    width: 100px; height: 100px; background-color:#FF0000;
    border: solid 1px black; margin: 10px;
    /*Flotar a la izquierda */
    float: left; }

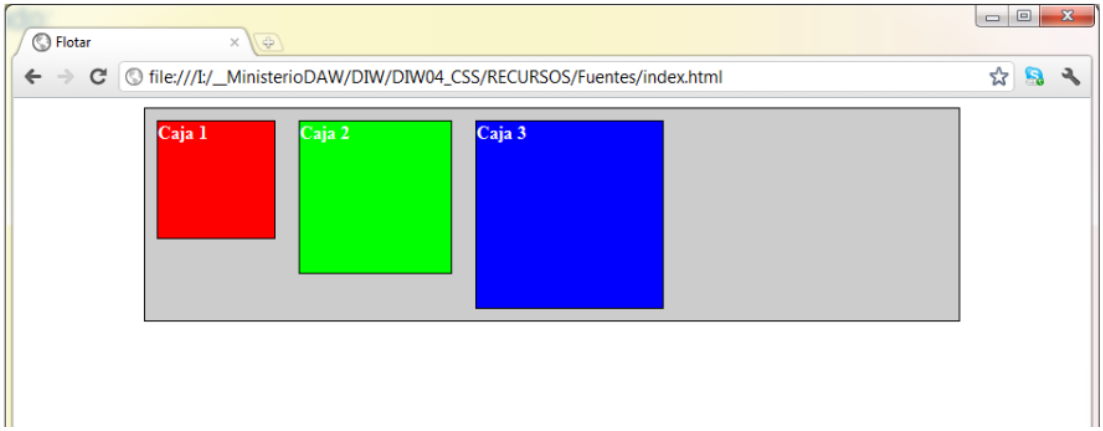
#caja2 {
    width: 130px; height: 130px; background-color:#00FF00;
    border: solid 1px black; margin: 10px;
    /*Flotar a la izquierda */
    float:left; }

#caja3 {
```

```
    width: 160px; height: 160px; background-color:#0000FF;
    border: solid 1px black; margin: 10px;
    /*Flotar a la izquierda */
    float: left; }

    /*Limpiar flotados a izquierda y derecha*/
    .clearboth {clear: both; }
```

La web con todas las cajas flotadas a la izquierda, y con un nuevo `div` que sirve para resetear los elementos flotados hasta este punto (donde se añade el `div`), quedaría del siguiente modo:



4.2.6. Bloque contenedor demasiado estrecho

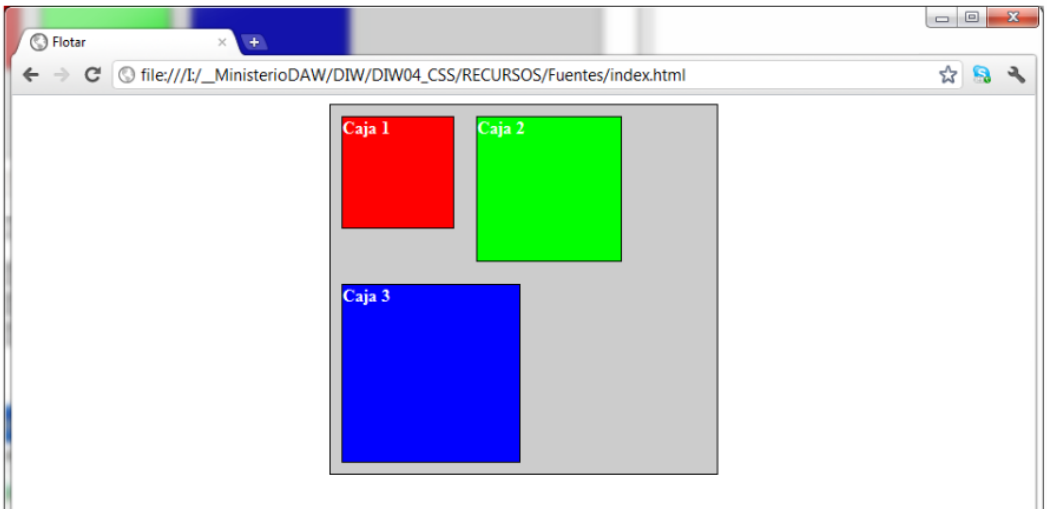
El caso a tratar es cuando nuestro `div` "contenido" es demasiado estrecho como para que los elementos flotados quepan horizontalmente. Si es así, el que no quepa se desplazará hacia abajo.

Podemos forzar esta situación configurando el ancho del `div` "contenido" con un valor de 350px (recordar que la suma de los anchos de las tres cajas, más sus márgenes, es mayor de 350px).

```
body { font-weight: bold; color: white; }

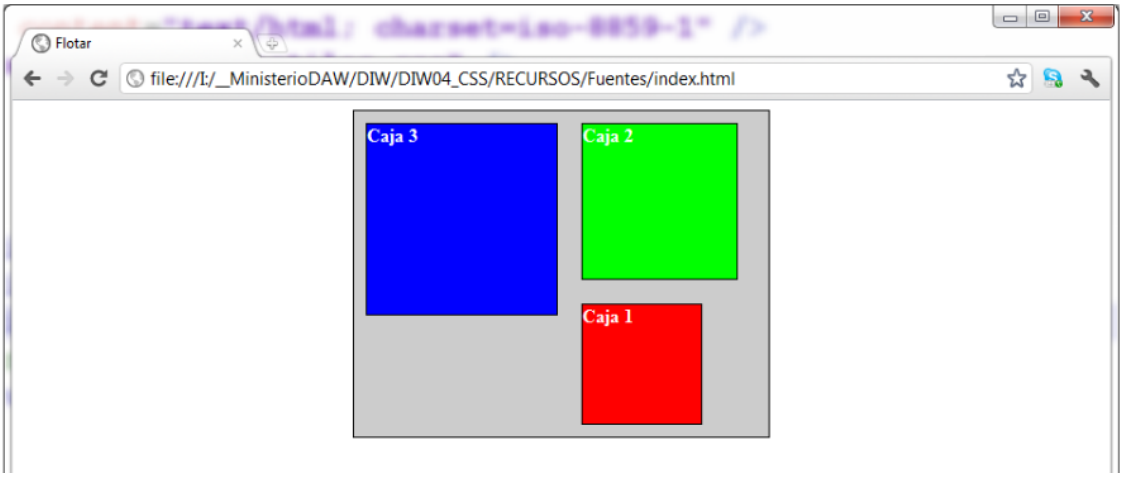
#contenido {
    /* Ancho de 350px y centrado en el navegador */
    width: 350px; margin: auto;
    /* Borde sólido y fondo gris claro */
    border: solid 1px black; background-color: #CCCCCC;}
```

La web con un bloque contenedor con un ancho de 350px, todas las cajas flotadas a la izquierda, y con el `div` que resetea los flotados hasta el punto donde se añade éste, se vería del siguiente modo:



El orden de las cajas en el documento HTML es Caja 1 – Caja 2 – Caja 3 pero, ¿cómo quedaría la web si cambiamos el orden de las cajas a Caja 3 – Caja 2 – Caja 1?

La web con el bloque contenedor de 350px, todas las cajas flotadas a la izquierda, y con el `div` que resetea los flotados hasta el punto donde se añade éste, pero con las cajas en el orden Caja 3 – Caja 2 – Caja 1, se vería del siguiente modo:



Podemos ver que el resultado cambia mucho. El motivo por el que no se pega la Caja 1 al borde del `div` "contenido" es porque la Caja1 sigue tropezando con la Caja 3 al intentar ser flotada a la izquierda.

4.3. clear

La propiedad `clear` se encarga de impedir elementos flotantes en la zona indicada, a la izquierda del elemento (`left`), a la derecha (`right`) o en ambos lados (`both`).

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
  <title>Document</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <ul>
    <li>Primer elemento</li>
    <li class="float">Segundo elemento</li>
    <li class="float">Tercer elemento</li>
    <li class="float fix">Cuarto elemento</li>
    <li class="float">Quinto elemento</li>
    <li class="float">Sexto elemento</li>
    <li>Séptimo elemento</li>
  </ul>
</body>
</html>
```

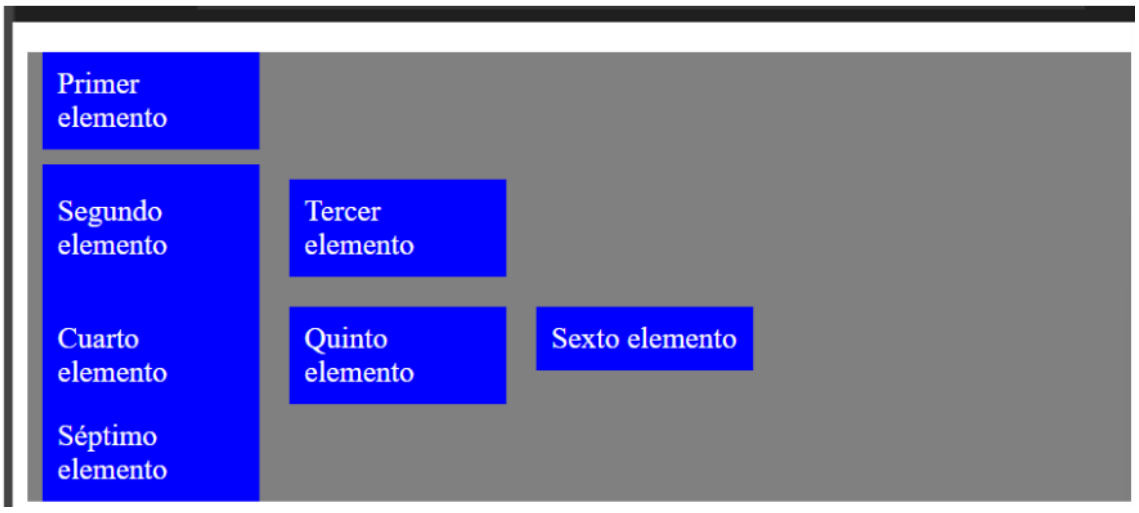
CSS:

```
ul {
  background-color: grey;
  padding: 0;
}

li{
  list-style-type: none;
    background: blue;
    width: 100px;
    padding: 8px;
    margin: 8px;
    color: white;
}

.float{
  float: left;
}

.fix {
  clear: both;
}
```



La propiedad `clear` puede tener los siguientes valores:

- `left` indica que el elemento comienza por debajo de cualquier otro elemento del bloque al que pertenece que estuviese flotando a la izquierda.
- `right` funciona como el `left` pero en este caso el elemento deberá estar flotando a la derecha.
- `both` mueve hacia abajo el elemento hasta que esté limpio de elementos flotantes a ambos lados.
- `none` permite elementos flotantes a ambos lados. Es el valor por defecto.
- `inherit` indica, al igual que en `float`, que heredará el valor de la propiedad `clear` de su elemento padre.

4.4. position

Aunque la mayoría de problemas pueden y deben ser solucionados con un modo u otro de la propiedad `display`, existen mecanismos alternativos y complementarios de posicionamiento que se utilizan para realizar acciones y casos muy concretos. Mediante la propiedad `position` podemos modificar la posición en donde aparecen los diferentes elementos y su contenido, al margen de utilizar sistemas como flex o grid.

La propiedad `position` se puede utilizar para modificar la posición donde aparecerá un elemento. Esta propiedad admite cinco valores:

1. `static`
2. `relative`
3. `absolute`
4. `fixed`
5. `sticky`

Si utilizamos un modo de posicionamiento diferente al estático (`absolute`, `fixed`, `sticky` o `relative`), podemos utilizar una serie de propiedades para modificar la posición de un elemento. Es importante tener en cuenta que dichas propiedades solo tendrán efecto si NO tenemos la propiedad position establecida a `static`.

Estas propiedades son las siguientes:

- `top`: Empuja el elemento una distancia desde la parte superior hacia el inferior.
- `bottom`: Empuja el elemento una distancia desde la parte inferior hacia la superior.
- `left`: Empuja el elemento una distancia desde la parte izquierda hacia la derecha.
- `right`: Empuja el elemento una distancia desde la parte derecha hacia la izquierda.
- `inset`: Propiedad de atajo para utilizar todas las anteriores.

Estas propiedades, lo único que hacen es colocar o fijar el elemento en el punto concreto indicado, de modo que si indicas `left: 0` significa que quieres que el elemento esté colocado 0 píxeles desde la izquierda. Puede contener valores negativos, invirtiendo la dirección.

4.4.1. static

`static` permite colocar los elementos según el flujo normal. Es el valor que asumirá por defecto en todos los elementos HTML.

4.4.2. relative

`relative` permite dejar el elemento exactamente donde está. Un elemento posicionado de esta forma se puede cambiar desde su punto de partida estableciendo para ello una distancia vertical y/o horizontal. En el siguiente ejemplo se desplaza la "caja2" 50px del extremo izquierdo y 50 px del extremo superior de su posición relativa.

```
#caja2 {width: 130px; height: 130px;
        background-color:#00FF00;
        border: solid 1px black;
        margin: 10px;
        /* Posicionamento relativo */
        position: relative;
        left: 50px; top: 50px; }
```

4.4.3. absolute

`absolute` permite abandonar el flujo normal del haciendo que el elemento no ocupe ningún espacio de forma que el resto de elementos del flujo normal actuarán como si el elemento no estuviese allí. El modo de determinar el origen de coordenadas de nuestro elemento será el siguiente:

- Si el elemento posicionado de modo absoluto no está contenido dentro de ningún otro, su origen de coordenadas se mide respecto a la esquina superior izquierda del `body` (contenedor principal).
- Si el elemento posicionado de modo absoluto está contenido dentro de otro elemento, el origen de coordenadas del elemento se calcula con respecto a la posición de la esquina superior izquierda del elemento que lo contiene.

4.4.4. fixed

`fixed` funciona de forma parecida al posicionamiento absoluto pero posiciona con respecto a la ventana del navegador apareciendo en la misma posición aunque el usuario se desplace por la página con las barras de desplazamiento.

4.5. visibility

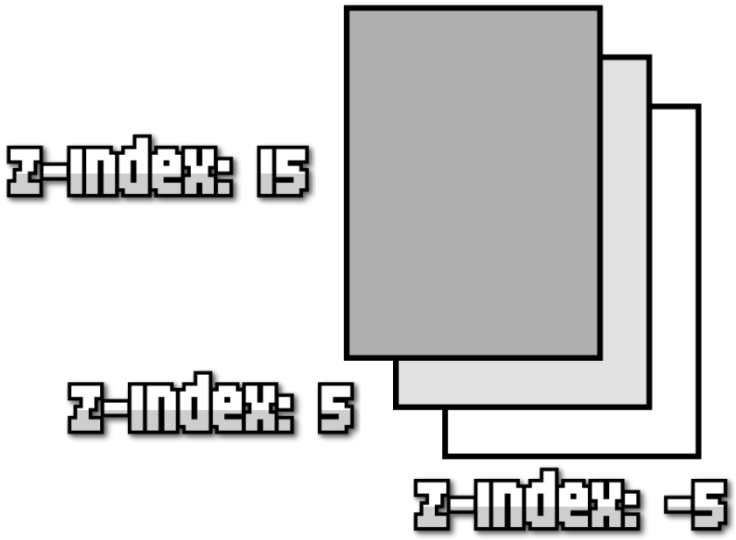
Esta propiedad controla si el elemento será visualizado según le asignes el valor `visible` o `hidden`. Debes tener en cuenta que, aunque un elemento no sea visible, éste continúa ocupando su espacio en el flujo normal del documento al contrario de lo que ocurría con la propiedad `display` cuando se le asignaba el valor `none`.

4.6. z-index

Permite controlar el orden en el que se presentan los elementos que quedan solapados por efecto de otras propiedades. Si cuando definimos algún elemento con posición absoluta, éste tiene que visualizarse en el mismo lugar ocupado por otro elemento, se producirá una superposición de elementos visualizándose, en la parte

coincidente, sólo el que está ocupando la "posición superior". La propiedad `z-index` permite especificar el orden en el eje z de los elementos, esto es, el orden de apilamiento en capas del documento.

Por defecto, los elementos se apilan en el orden en que aparecen: el elemento situado más abajo en el flujo normal del documento quedará encima. Si quieres que esta posición no sea tenida en cuenta, debes saber que los elementos con un valor mayor de la propiedad `z-index` son colocados encima de los que tienen un valor menor `z-index`, quedando estos últimos tapados por los primeros.

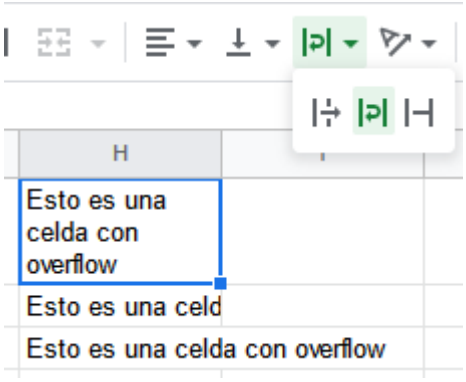


También debes saber que esta propiedad sólo se aplica a elementos que tengan la propiedad `position` en `absolute` o en `relative`.

4.7. overflow

La propiedad `overflow` define cómo se maneja el contenido que excede el tamaño del contenedor.

La propiedad `overflow` (desbordamiento en español) establece el comportamiento de desbordamiento. Es similar al menú de Google Sheets que se ve en la siguiente Figura.



Se puede definir tanto el overflow tanto en el eje X como en el eje Y con las propiedades overflow-x y overflow-y respectivamente.

4.8. clip-path

Hace algún tiempo, existió una propiedad CSS llamada clip, cuya intención era permitir recortes con ciertas formas como rectángulos. Sin embargo, hoy en día ha sido marcada como obsoleta, en favor de una nueva propiedad llamada clip-path.

5. CSS con tablas y listas

5.1. Cambiar apariencia de tablas

Para controlar la presentación de las tablas tenemos las propiedades:

- `caption-side`
- `table-layout`
- `border-collapse`
- `border-spacing`
- `empty-cells`
- `display`

5.1.1. caption-side

Esta propiedad sirve para indicar dónde se pone el título de la tabla. Puede tener los valores: `top`, `bottom`, `left` y `right`.

5.1.2. table-layout

Permite definir el modo en el que el navegador dibujará la tabla ya que puede hacerse de dos formas. Los valores que admite son:

- `fixed` dibuja la tabla basándose en las medidas establecidas en el código fuente. Con este valor se consigue que el sistema trabaje más rápido.
- `auto` dibuja la tablas basándose en el contenido de sus celdas. Es el valor por defecto.

5.1.3. border-collapse

Permite establecer el modo en el que se dibujan los bordes de las tablas: `separate` (separados), `collapse` (juntos) e `inherit`.

- En el modo `separate`, cada celda está rodeada por su borde haciendo el efecto de un borde con una línea doble.
- En el modo `collapse` las celdas contiguas comparten sus bordes.

5.1.4. border-spacing

Permite establecer la separación entre celdas contiguas. Para hacerlos se indica el valor del espaciado horizontal seguido del valor del espaciado vertical. Si se escribe un único valor, la separación horizontal y vertical serán iguales.

5.1.5. empty-cells

Esta propiedad soluciona la carencia que, al no dibujar las celdas que estaban vacías, obligaba a poner un espacio en blanco usando el carácter ` `. Los valores que admite son:

- `show` que permite mostrar los bordes y fondos como en las celdas con contenido.
- `hide` que permite ocultar los bordes y fondos de las celdas vacías.
- `inherit` que permite heredar el valor de empty-cells que tenga su elemento padre.

Ejemplos:

```
table{
  border:#000 solid thin;  /* Para que los bordes aparezcan */
  border-collapse: separate; /* Bordes separados */
  border-spacing: 0.5em 1em; /* Separación de los bordes */
  empty-cells: show;  /* Las celdas vacías se muestran */
}

table caption {
  caption-side: bottom;  /* El título de la tabla aparece debajo */
}
```

5.2. Cambiar apariencia de listas

Las listas son un elemento muy utilizado en las páginas web. Hoy en día, su empleo no está limitado a la simple enumeración de elementos en el contenido, también se utilizan para crear barras de navegación verticales y horizontales.

Para dar formato a las listas tenemos, entre otras, las propiedades:

- `list-style-type`
- `list-style-image`
- `list-style-position`
- `list-style`

5.2.1. list-style-type

Permite elegir el marcador visual de la lista asignando a la propiedad uno de los siguientes valores:

- `none` (eliminar el marcador)
- `square` (cuadrado)
- `disc` (círculo)
- `circle` (circunferencia)
- `lower-roman` (números romanos en minúscula)
- `lower-alpha` (letras en minúscula)
- `upper-alpha` (letras en mayúscula)
- etcétera.

5.2.2. list-style-image

Permite especificar una imagen como marcador. Para ello deberemos indicar la dirección o URL donde se encuentra la imagen.

Cuando se usa esta propiedad conviene declarar también la propiedad `list-style-type` en prevención de un fallo en la localización de la imagen.

Esto lo podríamos realizar, también empleando la propiedad `background` del elemento `li`. En este caso, mostrado en el ejemplo siguiente, debemos seguir los siguientes pasos:

1. Eliminar previamente el marcador visual estableciendo none como valor de la propiedad `list-style-type`.
2. Añadimos relleno a la izquierda de cada uno de los elementos de la lista.
3. Colocamos de nuevo el marcador visual, declarando la propiedad `background` a la que asignaremos la URL de una imagen.
4. Si cada elemento de la lista ocupa una sola línea, el marcador deberá centrarse verticalmente estableciendo su posición vertical al 50%.

Ejemplo:

```
ul {
  margin: 0;
  padding: 0;
  list-style-type: none;
}
li {
  background: url(boliche.gif) no-repeat 0 50%;
  padding-left: 20px;
}
```

5.2.3. list-style-position

Establece la posición del marcador de los elementos de la lista. Se puede colocar el marcador dentro del área de contenido con lo que todas las líneas de este elemento estarán alineadas por la izquierda (incluida la que lleva el marcador) o, se puede colocar fuera del área de contenido (como en una sangría francesa). Los valores que permiten posicionar el marcador son: `inside` (dentro) y `outside` (fuera).

5.2.4. list-style

Al igual que ocurría con otras propiedades que se vieron anteriormente, esta propiedad permite configurar las listas estableciendo, de forma abreviada y en cualquier orden, el valor de una o más de las propiedades individuales vistas en este apartado.

En el siguiente ejemplo se muestra el uso de las propiedades vistas hasta ahora.

Ejemplo:

```
ul {
  list-style-image: url(boliche.gif);
  list-style-type: disc;
  list-style-position: outside;
}

ul.especial {
  list-style: outside circle url("imagenes/balon.png");
}
```

Un efecto muy utilizado para convertir una lista en una barra de menú de navegación es colocar sus elementos dispuestos horizontalmente en la misma línea. El ejemplo siguiente muestra cómo hacerlo.

Ejemplo:

```
ul {
  list-style-type: none;
  padding: 0;
  border: 1px solid #000;
}

li {
  display: inline;
  border: 1px solid #00f;
}
```

Si nos interesa eliminar los bordes simplemente debemos poner la propiedad `border` a 0.